

STEMBridge Animation Engine - Project Context

Purpose of this document: give an LLM (Claude, Copilot, etc.) complete, accurate context about this codebase so it can generate new files, edits and shell commands that fit the existing architecture. Read all sections before answering. If something you need is not described here, ask for the file instead of guessing.

1. What the project is

A local-first, Manim-like educational animation engine for STEMBridge (a free, non-commercial STEM education initiative). Lessons for Class 10 Biology and Chemistry are authored in TypeScript, previewed in a browser, and rendered locally to MP4. Normal rendering uses no AI API, no cloud and no paid service. A finished prototype exists: the Diffusion lesson (biology), 40.1 s, 1920x1080, 30 fps, H.264.

- Audience: Class 10 students. Text must be large and readable. Theme: STEMBridge green, gold/yellow, white/cream (defined in src/core/theme.ts).
- Hard constraints: runs locally; TypeScript strict mode; reusable engine code separated from lesson code; deterministic timing; every lesson renderable from the command line; no AI API dependency in the renderer.
- Philosophy: build a minimal tested engine, then expand. Do not build a giant framework at once. A lesson describes WHAT happens; engine code knows HOW to draw/animate it.

2. Tech stack (versions as installed when built)

Package	Version	Role
pixi.js	8.21.0	All rendering (WebGL): shapes, text, particles, graph. Pixi v8 API.
vite	8.3.1	Dev server / bundling (ES modules, top-level await in main.ts)
typescript	7.0.2	Type checking only (noEmit), strict + noUnusedLocals/Parameters
playwright	1.63.0	Headless Chromium for frame capture
tsx	4.23.15	Runs TS scripts and tests in Node
@types/node	dev	Node typings for render/ and scripts/
FFmpeg	system	MP4 encoding, audio mixing (not an npm package)

Declared in the original brief but NOT installed yet (install only when a lesson needs them): d3, gsap, three, matter-js, katex. The engine uses its own Timeline instead of GSAP so that time is seekable and deterministic. Module system: ESM ("type": "module"), TS target ES2022, moduleResolution "bundler".

3. Directory structure

```
stembridge-animation-engine/  
  index.html           Vite entry; canvas host + preview controls (hidden with ?render)  
  package.json  tsconfig.json  vite.config.ts  README.md  
  src/  
    main.ts           Boots Renderer, builds lesson from ?lesson=, exposes window.__engine  
    types.d.ts        Global typing for window.__engine  
    core/  
      theme.ts        Colours, fonts, sizes, 1920x1080 canvas (ONLY place for styling constants)  
      Animation.ts    Easing functions + Track (stateless value-vs-time for one property)  
      Timeline.ts     Collection of tracks + updaters; seek(t) applies everything  
      Scene.ts        Authoring API (cursor-based, Manim-like)  
      Renderer.ts     Pixi Application wrapper; draws only on seek()/grab()  
      Assets.ts       loadFonts() (waits for document.fonts.ready)  
    components/  
      Text.ts Label.ts Panel.ts Arrow.ts Graph.ts Highlight.ts Reusable, lesson-agnostic drawables  
  biology/  
    Membrane.ts       Vertical membrane with pores (gaps)  
  simulation/  
    Diffusion.ts      PURE deterministic particle sim (no Pixi/DOM)  
    Particle.ts       ParticleField: draws many circles from a Float32Array  
  lessons/  
    index.ts          Registry: lesson name -> () => Scene  
    biology/diffusion.ts Prototype lesson (40.1 s)  
  render/  
    capture.ts        openLesson(): Vite server + Playwright Chromium; frameAt(page,t) -> PNG Buffer  
    render-scene.ts    CLI: still PNGs at given times
```

```

scripts/
  render-video.ts          CLI: all frames -> FFmpeg -> MP4 (resumable)
tests/
  engine.test.ts          node:test unit tests (timeline + simulation determinism)
out/                      Generated (stills, MP4, .frames cache). Do not commit.

```

Not built yet (planned in the brief): src/chemistry/ (Atom, Molecule, Bond, Reaction), src/biology/ (Cell, Organelle, DNA), src/components/Equation.ts (KaTeX), src/simulation/Osmosis.ts and Equilibrium.ts, lessons/chemistry/. Follow the same folder conventions when adding them.

4. The core idea: deterministic, seek-based animation

Nothing runs on a real-time clock and Pixi's ticker never runs (autoStart: false). Every visual property is a PURE FUNCTION OF TIME t (seconds). `Renderer.seek(t)` calls `Timeline.seek(t)`, then `app.render()`. Preview mode just calls `seek(elapsed)` in `requestAnimationFrame`; render mode calls `seek(frame / fps)` per frame. Same lesson + same seed = identical video.

- `Track (core/Animation.ts)`: ordered segments {start, duration, from, to, ease} for ONE numeric property. `valueAt(t)`: before the first segment -> first "from"; between segments -> previous "to"; after the last -> last "to". Stateless, so seeking backwards works.
- `Timeline.tween(target, prop, from, to, start, duration, ease = "easeInOut")` registers a segment on `target[prop]` (set via `Reflect.set`). `Timeline.updater(fn)` registers a callback `fn(t)` run on every seek - `fn` MUST be a pure function of t .
- `Timeline.restValue(target, prop, fallback)`: value a property will have after all queued tweens (used by `fadeOut/move` to start from the right value).
- Easings: linear, easeIn, easeOut, easeInOut.
- Custom components expose numeric properties designed for tweening (progress, phase) with setters that no-op when unchanged and redraw when changed.
- Simulations are PRECOMPUTED from a seed into arrays; `position(t)` is interpolation over the array. Never run live/random physics inside `seek()`.
- Never use `Math.random()`, `Date.now()`, `performance.now()` or GSAP/real-time tickers in lesson or engine code that affects frames. Use the seeded PRNG pattern in `simulation/Diffusion.ts` (mulberry32).

5. Authoring API: Scene (src/core/Scene.ts)

Scene owns root (a Pixi Container), timeline, and a time cursor. Every animation method starts at the cursor and ADVANCES the cursor by its duration. Objects with no `fadeOut` are visible from $t = 0$. `Scene.duration = max(cursor, timeline end)`.

Method	Behaviour
<code>add(node)</code>	Add any Pixi Container to the scene root; returns node
<code>addText(str, TextOptions) /</code> <code>addLabel(str, LabelOptions) /</code> <code>addArrow(ArrowOptions) /</code> <code>addGraph(GraphOptions)</code>	Factories; create the component and add it to the root
<code>title(str)</code>	Big centred title at $y=90$, fades in over 0.8 s
<code>fadeIn(node, dur=0.6) / fadeOut(node, dur=0.6)</code>	Tween alpha 0->1 / current->0
<code>write(textNode, dur=1)</code>	Left-to-right wipe via <code>Text.progress</code> . SINGLE-LINE text only (wrapped multi-line wipes badly; use <code>fadeIn</code>)
<code>reveal(arrowOrGraph, dur=1, ease="linear")</code>	Tween progress 0->1 (arrow draws out / graph lines draw in)
<code>move(node, {x?, y?}, dur=1)</code>	Tween position from its rest value
<code>highlight(rect, duration=2.5)</code>	Pulsing gold outline: fade in, pulse, fade out; returns Highlight
<code>wait(seconds)</code>	Advance cursor
<code>together(...actions)</code>	Run actions in parallel (all start at the same cursor); cursor moves to the longest

Method	Behaviour
simulate(start, duration, fn(localT))	Register a pure time callback active for [start, start+duration], localT clamped to [0, duration]
seek(t)	Apply all tracks/updaters at time t (called by Renderer)

Coordinate system: Pixi stage 1920 x 1080 px, origin top-left, y downward. Text is anchored at its centre by default (align: "left" anchors at left-middle). Graph and Panel use ABSOLUTE scene coordinates (container at 0,0).

6. Components and simulation classes

Class (file)	Constructor / key API
Text (components/Text.ts)	new Text(str, {x, y, size?=46, color?, bold?, align?: "center" "left", maxWidth?}); numeric prop progress (0..1) wipe-reveal
Label (components/Label.ts)	new Label(str, {x, y, size?=38, fill?, border?, color?}); rounded chip centred on (x,y)
Panel (components/Panel.ts)	new Panel({x, y, w, h, fill?, border?, radius?}); white rounded rectangle with green border
Arrow (components/Arrow.ts)	new Arrow({from:[x,y], to:[x,y], color?, width?}); prop progress (0..1)
Highlight (components/Highlight.ts)	new Highlight({x,y,w,h}); prop phase (radians) drives pulse; also exports type Rect {x,y,w,h}
Graph (components/Graph.ts)	new Graph({x,y,width,height,title,xLabel,yLabel,xMax,yMax,xTicks,yTicks,yUnit?,series:[{label,color,points:[x,y][][]}]}); prop progress reveals lines up to progress*xMax; methods px(x), py(y), region(x0,x1) -> Rect in scene coords
Membrane (biology/Membrane.ts)	new Membrane({x, top, bottom, pores:[{y,h}], thickness?=20}); dark-green vertical bar with gaps
ParticleField (simulation/Particle.ts)	new ParticleField(radius=10, fill?, outline?); setPositions(Float32Array [x0,y0,x1,y1,...]) redraws all circles
DiffusionSim (simulation/Diffusion.ts)	new DiffusionSim({seed,count,bounds,membraneX,pores,speed,turnRate,radius,duration,dt?=1/60}); positionsAt(t,out) fills Float32Array; leftFractionAt(t) in 0..1; equilibriumTime(tol). Pure TS, no Pixi: unit-testable in Node.
Renderer (core/Renderer.ts)	await Renderer.create(hostEl); show(scene); seek(t); grab(t) -> base64 PNG of canvas

theme (core/theme.ts): width 1920, height 1080, font stack, colors {bg, panel, green, greenDark, greenLight, gold, goldDark, goldLight, ink, muted} as 0xRRGGBB numbers, size {title 88, body 46, label 38, small 30}. Always take colours and sizes from theme; never hard-code them in lessons.

Diffusion prototype parameters (src/lessons/biology/diffusion.ts): box {x:120,y:240,w:1000,h:600}; membrane at x=620 with 3 pores of height 100; 150 particles, seed 7, speed 450, turnRate 1, radius 10; 16 s of simulated time; graph of left/right concentration (%) driven by the same simulation; equilibrium time computed via sim.equilibriumTime(0.07); ends with a summary slide of three bullets.

7. Rendering pipeline

```
Lesson (TypeScript, returns Scene)
-> Vite dev server (programmatic, started by render/capture.ts)
-> headless Chromium via Playwright, page URL ?lesson=<name>&render
-> for frame f: page.evaluate(window.__engine.grab(f / fps)) (seek + render + canvas.toDataURL)
-> PNG written to out/.frames/<lesson>-<fps>fps/000000.png (existing frames are skipped = resumable)
-> ffmpeg -framerate fps -i %06d.png [-i audio -af apad -c:a aac] -c:v libx264 -pix_fmt yuv420p -crf 18
-> out/<lesson>.mp4 (frames folder deleted unless --keep-frames)
```

main.ts reads URL params: ?lesson=<name> (default "diffusion") and ?render (adds body.render: hides controls, canvas fixed at 1920x1080). It sets window.__engine = {ready, duration, seek(t), grab(t)}. Playwright launches Chromium with SwiftShader WebGL flags; set env CHROME_PATH to use a specific Chrome/Edge binary.

8. Commands

```
npm install
npx playwright install chromium # or: export CHROME_PATH=/path/to/chrome
npm run dev # preview: http://127.0.0.1:5173/?lesson=diffusion
npm run render -- <lesson> --t 0,10,24 # still PNGs -> out/stills/
npm run video -- <lesson> # -> out/<lesson>.mp4 (1920x1080, 30 fps)
npm run video -- <lesson> --fps 60 --audio narration.mp3 --out out/x.mp4 --keep-frames
npm test # node:test via tsx (timeline + sim determinism)
npm run typecheck # tsc --noEmit
```

9. How to add a new lesson (checklist)

- Create src/lessons/<subject>/<name>.ts exporting function <name>(): Scene.
- Register it in src/lessons/index.ts: lessons["<name>"] = <name>. The key is the CLI name.
- Need a new visual building block? Put it in src/components (generic), src/biology or src/chemistry (domain), or src/simulation (physics/particles). Do not put drawing code inside a lesson file.
- Need physics/randomness? Write a pure, seeded, precomputed class in src/simulation with positionsAt(t) style accessors and add a unit test in tests/.
- Verify: npm run typecheck, npm test, npm run render -- <name> --t ..., then npm run video -- <name>.

```
import { Container } from 'pixi.js';
import { Scene } from '../core/Scene';
import { theme } from '../core/theme';

export function myLesson(): Scene {
  const scene = new Scene();
  const c = theme.colors;

  // Group content so it can fade out as one unit
  const content = scene.add(new Container());
  const put = <T extends Container>(n: T): T => content.addChild(n);

  scene.title('My Topic'); // fades in at top
  const text = put(scene.addText('One short line', { x: 960, y: 500, size: 52 }));
  scene.write(text, 1.2); // single-line wipe
  const chip = put(scene.addLabel('Key term', { x: 960, y: 650 }));
  scene.together(() => scene.fadeIn(chip, 0.6), () => scene.wait(1));
  scene.highlight({ x: 700, y: 600, w: 520, h: 100 }, 2); // gold pulse, then fades
  scene.fadeOut(content, 0.8);
  scene.wait(1);
  return scene; // duration = max(cursor, timeline end)
}
// Then register in src/lessons/index.ts: 'my-lesson': myLesson
```

10. Coding conventions and gotchas

- TypeScript strict; noUnusedLocals/noUnusedParameters are ON (unused imports fail typecheck). isolatedModules is on: use "import type" for type-only imports. Relative imports without extensions.
- Pixi v8 Graphics API is chained shape-then-style: g.roundRect(x,y,w,h,r).fill(color).stroke({width,color}); lines: g.moveTo().lineTo().stroke(...). Text: new Text({text, style:{...}}) with .anchor.set(). Do not use the old v7 beginFill/drawRect API.
- Custom animatable properties must be plain numeric getters/setters on the component so Timeline.tween(node, "prop", ...) type-checks (NumericKeys).
- Fade groups: put many nodes in one Container and fadeOut that container.
- Keep text large (>= 38 px for labels, 46+ for captions). Wrapped text needs maxWidth and align:"left"; reveal it with fadeIn, not write().
- Fonts are system fonts by default, so glyph widths differ by machine. For identical output bundle a font (e.g. @fontsource/inter) and set theme.font.
- Rendering is software WebGL (SwiftShader) in headless mode: slower than GPU (about 0.5 s/frame at 1080p on one CPU core) but reproducible. Video renders are resumable.
- out/ and node_modules/ are generated; do not commit them (add to .gitignore).

11. How an LLM should respond to requests for this project

- The owner (Aarish) sends a prompt (e.g. "add an Osmosis lesson" or "add a Cell component"). Reply with COMPLETE file contents for every new or changed file (path as heading, then full code), never fragments with "..." placeholders.

- Then give the exact terminal commands to run (install steps if new dependencies, typecheck, test, render a still, render the video) in the order to run them. State clearly which commands are new.
- Respect the architecture: engine vs lesson separation, determinism rules (section 4), theme constants, strict TypeScript. Prefer extending existing classes over rewriting them.
- Keep new dependencies minimal and local-only; never add anything that needs an AI API or paid service at render time.
- If a change touches an existing file, show the whole updated file or a precise search/replace, and mention which other files import it.
- If unsure about the current content of a file, ask the owner to paste it rather than guessing.